

iOS 广告SDK接入文档

前提条件

- 推荐Xcode 12及以上版本
- 支持iOS 13.0或更高版本

导入 SDK

使用CocoaPods导入SDK

```
pod 'PTGAdFramework', '2.3.12'
```

Topon支持

SDK 在2.0.7之后的版本支持Topon的聚合请求广告（支持信息流，开屏，横幅），可在Topon后台配置广告请求的适配器，具体可联系Fancy 商务配置,技术接入需导入

AnyThinkPTGAdSDKAdapter 具体可参照 Demo中Topon文件夹中相关的代码

```
pod 'AnyThinkPTGAdSDKAdapter', '1.2.3'
pod 'PTGAdFramework', '2.3.12'

# topon 适配器
插屏  ATPTGInterstitialAdapter
横幅  ATPTGNativeExpressBannerAdapter
原生  ATPTGNativeExpressAdapter      #支持自渲染需要在topon后台配置参数 self_render,值等于1
开屏  ATPTGSplashAdapter
激励  ATPTGRewardedVideoAdapter
```

GroMore适配器支持

需要在GroMore后台添加自定义广告平台

初始化适配器

PTGGMConfig

需要在网络配置中填写AppId AppKey

开屏

PTGGMSplashAdapter

信息流

PTGGMNativeAdapter

信息流 信息流 支持自渲染和模板渲染，在GroMore后台添加广告位下的代码位时，选择渲染类型 自渲染广告

插屏

PTGGMInterstitialAdapter

横幅

PTGGMBannerAdapter

激励

PTGGMRewardedVideoAdapter

ToBid适配器

初始化

PTGToBidConfigAdapter

需要在添加自定义广告源时，需要添加应用维度参数app_id,app_key参数（fancy广告平台的app_id,app_key）

开屏

PTGToBidSplashAdapter

信息流

PTGToBidNativeAdapter

支持模板及自渲染信息流 默认模板，如需支持自渲染需在添加广告源代码位界面时，自定义配置信息中填写 banner

PTGToBidBannerAdapter

插屏

PTGToBidInterstitialAdapter

插屏默认点击广告后不关闭，如果需要点击后关闭插屏，可以在tobid自定义配置中填写 {"closeAfterClick":true}

激励

PTGToBidRewardedVideoAdapter

极光适配器

在 2.2.72版本之后支持极光聚合广告，接入方式参照 [github链接](#)

将SKAdNetwork ID 添加到 info.plist 中，以保证 SKAdNetwork 的正确运行

```
<key>SKAdNetworkItems</key>
<array>
  <dict>
    <key>SKAdNetworkIdentifier</key>
    <string>238da6jt44.skadnetwork</string>
  </dict>
  <dict>
    <key>SKAdNetworkIdentifier</key>
    <string>22mmun2rn5.skadnetwork</string>
  </dict>
</array>
```

Info.plist 配置说明

为了保证 广告投放的精准性，在接入 SDK 时需要在 App 的 Info.plist 中添加 LSAApplicationQueriesSchemes 配置。

```
<key>LSApplicationQueriesSchemes</key>
<array>
  <string>openApp.jdMobile</string>
  <string>tbopen</string>
  <string>tmall</string>
  <string>imeituan</string>
  <string>OneTravel</string>
  <string>pinduoduo</string>
  <string>alipays</string>
  <string>iosamap</string>
  <string>kwai</string>
  <string>ksnebula</string>
  <string>cainiao</string>
  <string>youku</string>
  <string>taobao</string>
  <string>baiduboxapp</string>
  <string>baiduboxlite</string>
  <string>ddxq</string>
  <string>fleamarket</string>
  <string>douyu</string>
  <string>dangdang</string>
  <string>hellobike</string>
  <string>zhihu</string>
  <string>taobaotravel</string>
  <string>qklink</string>
```

```
<string>xmly</string>
<string>dianping</string>
<string>soul</string>
<string>sinaweibo</string>
<string>jdjr</string>
<string>weixin</string>
<string>tyc</string>
<string>eleme</string>
<string>upcommon</string>
<string>snssdk1128</string>
<string>snssdk2329</string>
<string>vipshop</string>
<string>autohome</string>
<string>qqcar</string>
<string>mcdonalds</string>
</array>
```

全局配置

配置ali_aaid

媒体端获取到ali_aaid 通过一下方法设置

```
[PTGSDKManager setAdIdentifier:@"you aaid" ];
```

应用传输安全

应用传输安全(ATS) 是 iOS 9 中引入的隐私设置功能。默认情况下，系统会为新应用启用该功能，并强制实施安全连接。为确保您的广告不受 ATS 影响，请执行以下操作：

在您的应用的 Info.plist 文件中，添加 NSAllowsArbitraryLoads 以停用 ATS 限制。

```
<key>NSAppTransportSecurity</key>
<dict>
    <key>NSAllowsArbitraryLoads</key>
    <true/>
</dict>
```

iOS 14 AppTrackingTransparency

iOS 14 开始，在应用调用 App Tracking Transparency framework 向用户请求应用跟踪权限之前，IDFA 将不可用。如果应用没有向用户发出请求，则 IDFA 将自动清零，这可能会导致广告收入出现重大损失。因为用户必须授权IDFA。

应用开发者可以控制请求 App Tracking Transparency 框架来申请跟踪权限的时机。

要显示用于访问 IDFA 的 App Tracking Transparency 授权请求，请更新 Info.plist 以添加 `NSUserTrackingUsageDescription` 键，值为描述应用如何使用 IDFA 的自定义消息。以下是示例描述文字：

```
<key>NSUserTrackingUsageDescription</key>
<string>需要获取您设备的广告标识符，以为您提供更好的广告体验</string>
```

要显示授权请求对话框，请调用 `requestTrackingAuthorizationWithCompletionHandler:`。我们建议您在授权回调之后再加载广告，以便如果用户允许跟踪权限，则 SDK 可以在广告请求中使用 IDFA，示例代码如下：

```
#import <AppTrackingTransparency/AppTrackingTransparency.h>
#import <AdSupport/AdSupport.h>

- (void)requestIDFA {
    if (@available(iOS 14, *)) {
        [ATTrackingManager requestTrackingAuthorizationWithCompletionHandler:^(ATTrackingManagerAuthorizationStatus status) {
            dispatch_async(dispatch_get_main_queue(), ^{
                // 加载广告
            });
        }];
    } else {
        // 加载广告
    }
}
```

SKAdNetwork (SKAN) 是 Apple 的归因解决方案，可帮助广告客户在保持用户隐私的同时衡量广告活动。使用 Apple 的 SKAdNetwork 后，即使 IDFA 不可用，广告网络也可以正确获得应用安装的归因结果。访问 <https://developer.apple.com/documentation/storekit/skadnetwork> 了解更多信息。

1. 应用编译环境升级至 Xcode 12.0 及以上版本
2. 将 SKAdNetwork ID 添加到 info.plist 中，以保证 SKAdNetwork 的正确运行

```
<key>SKAdNetworkItems</key>
<array>
    <dict>
        <key>SKAdNetworkIdentifier</key>
        <string>238da6jt44.skadnetwork</string>
```

```
</dict>
<dict>
  <key>SKAdNetworkIdentifier</key>
  <string>22mmun2rn5.skadnetwork</string>
</dict>
</array>
```

SDK初始化

加载广告之前，请先使用 PTGAdSDK 应用 ID 进行初始化，此操作仅需执行一次，最好是在应用启动时执行。

```
#import <PTGAdSDK/PTGAdSDK.h>
@implementation AppDelegate

- (BOOL)application:(UIApplication *)application
    didFinishLaunchingWithOptions:(NSDictionary *)launchOptions {
    /// 获取PTGAdSDK版本号
    NSString *sdkVersion = [PTGAdSDKManager getSDKVersion];

    /// 配置跟踪id
    /// 重要 影响广告填充
    /// 避免代码中明文出现caid ali_id等字符 审核相关
    [PTGSDKManager setAdIds:@{
        @"idfa":idfa,
        @"one_id":caid,
        @"one_id_version": caidVersion,
        @"last_id": lastCaid,
        @"last_id_version": lastCaidVersion,
        @"one_ali_id": ali_aaid
    }];

    /// appKey Ptg后台创建的媒体ID
    /// appSecret Ptg后台创建的媒体密钥
    [PTGSDKManager syncSetAppKey:@"45271" appSecret:@"Y6yyc3zyP3E09o18"];
    return YES;
}

@end
```

开屏广告

开屏广告用于当用户进入您的应用展示，一般会展示 5 秒（部分可以跳过），倒计时结束会自动关闭。与插屏广告不同，开屏广告可以轻松地展示应用品牌（Logo 和名称），以便用户了解他们看到广告的环境。

开屏广告加载

初始化开屏广告对象，使用开屏广告id加载广告。

以下示例演示了如何在 AppDelegate 的 application:didFinishLaunchingWithOptions: 方法中创建并请求开屏广告。

```
#import <PTGAdSDK/PTGAdSDK.h>

@interface AppDelegate () <PTGSplashAdDelegate>
@property (nonatomic, strong) PTGSplashAd *splashAd;
@end

@implementation AppDelegate

- (BOOL)application:(UIApplication *)application didFinishLaunchingWithOptions:(NSDictionary *)launchOptions {
    self.window = [[UIWindow alloc] initWithFrame:[UIScreen mainScreen].bounds];
    self.window.backgroundColor = [UIColor whiteColor];
    self.window.rootViewController = [[UINavigationController alloc] initWithRootViewController:PTGViewController];
    [self.window makeKeyAndVisible];

    /// 获取sdk version
    NSString *version = [PTGSDKManager getSDKVersion];

    /// appKey Ptg后台创建的媒体ID
    /// appSecret Ptg后台创建的媒体密钥
    [PTGSDKManager setAppKey:@"45271" appSecret:@"Y6yyc3zyP3E09o18"
    completion:^(BOOL result, NSError *error) {
        if (result) {
            [self.splashAd loadAd];
        }
    }];

    return YES;
}

#pragma mark - get -
- (PTGSplashAd *)splashAd {
    if (!_splashAd) {
        UIView *bottomView = [[UIView alloc] initWithFrame:CGRectMake(0, 0, [[UIScreen mainScreen] bounds].width, 50)];
        UIImageView *logo = [[UIImageView alloc] initWithImage:[UIImage imageNamed:@"SplashLogo"]];
    }
}
```

```

        logo.accessibilityIdentifier = @"splash_logo";
        logo.frame = CGRectMake(0, 0, 311, 47);
        logo.center = bottomView.center;
        [bottomView addSubview:logo];

        _splashAd = [[PTGSplashAd alloc] initWithPlacementId:@"900000228"];
        _splashAd.delegate = self;
        _splashAd.rootViewController = [UIApplication sharedApplication].windows.firstObject.rootViewCon
        _splashAd.bottomView = bottomView;
    }
    return _splashAd;
}
@end

```

开屏广告事件

设置开屏广告的delegate，delegate遵守并实现PTGSplashAdDelegate，可以监听广告的生命周期事件。

```

#pragma mark - PTGSplashAdDelegate -
/// 开屏加载成功
- (void)ptg_splashAdDidLoad:(PTGSplashAd *)splashAd {
    NSLog(@"开屏广告%s", __func__);
    /// 广告是否有效（展示前请务必判断）
    /// 如不严格按照此方法对接，将导致因曝光延迟时间造成的双方消耗gap过大，请开发人员谨慎对接
    if (splashAd.isReady) {
        [splashAd showAdWithViewController:@"当前的栈顶控制器"];
    } else {
        NSLog(@"广告已过期");
    }
}

/// 开屏加载失败
- (void)ptg_splashAd:(PTGSplashAd *)splashAd didFailWithError:(NSError *)error {
    NSLog(@"开屏广告%s", __func__);
}

/// 开屏广告被点击了
- (void)ptg_splashAdDidClick:(PTGSplashAd *)splashAd {
    NSLog(@"开屏广告%s", __func__);
}

/// 开屏广告关闭了

```



```

- (void)ptg_splashAdDidClose:(PTGSplashAd *)splashAd {
    NSLog(@"开屏广告%s",__func__);
}

/// 开屏广告将要展示
- (void)ptg_splashAdWillVisible:(PTGSplashAd *)splashAd {
    NSLog(@"开屏广告%s",__func__);
}

/// 开屏广告展示失败
- (void)ptg_splashAdVisibleError:(PTGSplashAd *)splashAd error:(NSError *)error {
    NSLog(@"开屏广告展示失败%s error = %@",__func__,error);
}

```

信息流广告

信息流广告是通过平台原本就有的界面组件向用户呈现的广告素材资源。这种广告使用您在构建布局时已经采用的同类视图进行展示，而且能以和周围视觉设计相称的形式呈现，让用户有浑然一体的使用体验。从代码编写的角度来说，这意味着当信息流广告加载时，由应用负责展示它们了。本指南向您介绍如何将信息流广告集成到 iOS 应用中。

信息流广告加载

信息流广告通过PTGNativeExpressAdManager对象来加载信息流广告。以下示例演示了如何在UIViewController 的 viewDidLoad 方法中创建 并请求信息流广告。

普通信息流：

```

#import <PTGAdSDK/PTGAdSDK.h>

@interface PTGNativeExpressFeedViewController ()<PTGNativeExpressAdDelegate,UITableViewDelegate,UITableViewDataSource>

@property(nonatomic,strong)PTGNativeExpressAdManager *manager;
@property(nonatomic,strong)UIButton *loadButton;
@property(nonatomic,strong)UITableView *tableView;
@property(nonatomic,strong)NSArray<PTGNativeExpressAd *> *ads;

@end

- (void)viewDidLoad {
    [super viewDidLoad];
    self.view.backgroundColor = [UIColor lightGrayColor];
    [self.manager loadAd];
}

```

```

#pragma mark - get -
- (PTGNativeExpressAdManager *)manager {
    if (!_manager) {
        /// 普通信息流广告高度设置为0 广告渲染成功后, 获取广告视图的实际高度
        _manager = [[PTGNativeExpressAdManager alloc] initWithPlacementId:@"900000231"
                                                                type:PTGNativeExpressAdTypeFeed
                                                                adSize:CGSizeMake(self.view.bounds.size.width, 0)];

        _manager.delegate = self;
    }
    return _manager;
}

```

自渲染信息流:

```

#import <PTGAdSDK/PTGAdSDK.h>

@interface PTGNativeExpressFeedViewController ()<PTGNativeExpressAdDelegate,UITableViewDelegate,UITableViewDataSource>

@property(nonatomic,strong)PTGNativeExpressAdManager *manager;
@property(nonatomic,strong)UIButton *loadButton;
@property(nonatomic,strong)UITableView *tableView;
@property(nonatomic,strong)NSArray<PTGNativeExpressAd *> *ads;

@end

- (void)viewDidLoad {
    [super viewDidLoad];
    self.view.backgroundColor = [UIColor lightGrayColor];
    [self.manager loadAd];
}

#pragma mark - get -
- (PTGNativeExpressAdManager *)manager {
    if (!_manager) {
        /// 普通信息流广告高度设置为0 广告渲染成功后, 获取广告视图的实际高度
        _manager = [[PTGNativeExpressAdManager alloc] initWithPlacementId:@"900000231"
                                                                type:PTGNativeExpressAdTypeSelfRender
                                                                adSize:CGSizeMake(self.view.bounds.size.width, 0)];

        _manager.delegate = self;
    }
    return _manager;
}

///自渲染广告对象的渲染方法

```

```

- (void)renderAd:(PTGNativeExpressAd *)ad {
    _ad = ad;
    [ad darwUnregisterView];
    self.titleLabel.text = ad.adObject.adData.title;
    self.bodyLabel.text = ad.adObject.adData.body;

    self.iv.hidden = ad.isVideoAd;

    if (ad.isVideoAd) {
        if (self.relatedView.videoView.superview == self) {
            [self.relatedView.videoView removeFromSuperview];
        }
        self.relatedView = ad.adObject.relatedView;
        [self addSubview:self.relatedView.videoView];
        [self.relatedView.videoView mas_makeConstraints:^(MASConstraintMaker *make) {
            make.edges.equalTo(self.iv);
        }];
        self.relatedView.videoDelegate = self;
        PTGMediaInfo *info = ad.adObject.adData.videoAdInfo;
        NSLog(@"当前素材宽 = %d 高 = %d",info.width,info.height);
    } else {
        [self.relatedView.videoView removeFromSuperview];
        self.relatedView = nil;
        PTGMediaInfo *info = ad.adObject.adData.imageUrls.firstObject;
        NSLog(@"当前素材宽 = %d 高 = %d",info.width,info.height);
        NSURL *url = [NSURL URLWithString:ad.adObject.adData.imageUrls.firstObject.url];
        [self.iv sd_setImageWithURL:url];
    }
    [self addSubview:self.closeButton];
    [ad.adObject setContainer:self clickableViews:@[self]];
}

```

在2.2.90中,信息流广告对象接口调整, nativeExpressAdView为广告返回的模板视图, 渲染成功后, 需要将nativeExpressAdView添加到广告容器视图中。

adObject为自渲染广告对象, 自渲染时使用。

具体接入参考的demo中ViewController/Native文件夹中的示例

```

/// 信息流广告对象
@interface PTGNativeExpressAd : NSObject

/// 自渲染广告对象, 自渲染时使用
@property(nonatomic,strong) PTGNativeAdObject *adObject;

/// 模板视图 模板广告时使用

```

```

@property(n nonatomic, strong, readonly) UIView *nativeExpressAdView;
/// 是否是模板广告 false 为自渲染
@property(n nonatomic, assign) BOOL isNativeExpress;
/// 广告是否有效 (展示前请务必判断)
/// 如不严格按照此方法对接, 将导致因曝光延迟时间造成的双方消耗gap过大, 请开发人员谨慎对接
@property(n nonatomic, assign, readonly) BOOL isReady;
/// 是否是视频广告 只对自渲染视频有效
@property(n nonatomic, assign, readonly) BOOL isVideoAd;
/// 详解: [必选]开发者需传入用来弹出目标页的ViewController, 一般为当前ViewController
@property(n nonatomic, weak) UIViewController *controller;
/// 价格 单位分
@property(n nonatomic, assign) NSInteger price;
/// 消耗方类型
@property(n nonatomic, assign) PTGAdSourceType sourceType;
/// [必选]原生模板广告渲染
- (void)render;
/// 移除注册视图
- (void)darwinUnregisterView;

/// 广告素材 可能为空 自渲染时请使用PTGNativeAdObject
@property(n nullable, nonatomic, strong) PTGAdMaterial *adMaterial;

@end

```

信息流广告事件

设置信息流广告的delegate, delegate遵守并实现PTGNativeExpressAdDelegate, 可以监听广告的生命周期事件。

在信息流广告加载成功后会返回PTGNativeExpressAd的广告对象, 调用PTGNativeExpressAd对象的render方法, 并设置PTGNativeExpressAd的controller, 调用PTGNativeExpressAd的render方法后, 在渲染成功的方法中刷新数据, 详情见以下示例代码

```

#pragma mark - PTGNativeExpressAdDelegate -
/// 原生模版广告获取成功
/// @param manager 广告管理类
/// @param ads 广告数组 一般只会有一条广告数据 使用数组预留扩展
- (void)ptg_nativeExpressAdSuccessToLoad:(PTGNativeExpressAdManager *)manager ads:(NSArray<__kindof PTGNativeExpressAd> *)ads {
    NSLog(@"信息流广告获取成功, %@", ads);

    [ads enumerateObjectsUsingBlock:^(__kindof PTGNativeExpressAd * _Nonnull obj, NSUInteger idx, BOOL *stop) {
        [obj render];
        [obj setController:self];
    }];
}

```

```

        NSMutableArray *arrM = self.ads ? self.ads.mutableCopy : [NSMutableArray new];
        [arrM addObjectsFromArray:ads];
        self.ads = arrM.copy;
    }

    /// 原生模版广告获取失败
    /// @param manager 广告管理类
    /// @param error 错误信息
    - (void)ptg_nativeExpressAdFailToLoad:(PTGNativeExpressAdManager *)manager error:(NSError *_Nullable)error {
        NSLog(@"信息流广告加载失败, %@", error);
    }

    /// 原生模版渲染成功
    /// @param nativeExpressAd 渲染成功的模板广告
    - (void)ptg_nativeExpressAdRenderSuccess:(PTGNativeExpressAd *)nativeExpressAd {
        NSLog(@"信息流广告渲染成功, %@", nativeExpressAd);
        /// 广告是否有效 (展示前请务必判断)
        /// 如不严格按照此方法对接, 将导致因曝光延迟时间造成的双方消耗gap过大, 请开发人员谨慎对接
        if (!nativeExpressAd.isReady) {
            NSLog(@"信息流广告已过期, %@", nativeExpressAd);
            NSMutableArray *ads = [self.ads mutableCopy];
            [ads removeObject:nativeExpressAd];
            self.ads = ads;
        }
        [self.collectionView reloadData];
    }

    /// 原生模版渲染失败
    /// @param nativeExpressAd 渲染失败的模板广告
    /// @param error 渲染过程中的错误
    - (void)ptg_nativeExpressAdRenderFail:(PTGNativeExpressAd *)nativeExpressAd error:(NSError *_Nullable)error {
        NSLog(@"信息流广告渲染失败, %@", error);
        NSMutableArray *arrM = self.ads.mutableCopy;
        [arrM containsObject:nativeExpressAd] ? [arrM removeObject:nativeExpressAd] : nil;
        [self.collectionView reloadData];
    }

    /// 原生模板将要显示
    /// @param nativeExpressAd 要显示的模板广告
    - (void)ptg_nativeExpressAdWillShow:(PTGNativeExpressAd *)nativeExpressAd {
        NSLog(@"信息流广告曝光");
    }

    /// 广告显示失败, 广告资源过期 (媒体缓存广告, 广告展示时, 广告资源已过期)

```

```

/// @param nativeExpressAd 展示失败的广告
/// 展示失败后，请移除广告，如不严格按照此方法对接，将导致因曝光延迟时间造成的双方消耗gap过大，请开发人员谨慎对接
- (void)ptg_nativeExpressAdShowFail:(PTGNativeExpressAd *)nativeExpressAd error:(NSError *_Nullable)error {
    NSLog(@"信息流广告曝光失败 error = %@",error);
}

/// 原生模板将被点击了
/// @param nativeExpressAd 被点击的模板广告
- (void)ptg_nativeExpressAdDidClick:(PTGNativeExpressAd *)nativeExpressAd {
    NSLog(@"信息流广告被点击");
}

/// 原生模板广告被关闭了
/// @param nativeExpressAd 要关闭的模板广告
- (void)ptg_nativeExpressAdViewClosed:(PTGNativeExpressAd *)nativeExpressAd {
    NSLog(@"信息流广告将要被关闭");
}

/// 原生模板广告将要展示详情页
/// @param nativeExpressAd 广告
- (void)ptg_nativeExpressAdWillPresentScreen:(PTGNativeExpressAd *)nativeExpressAd {
    NSLog(@"信息流广告展示详情页");
}

/// 原生模板广告将要关闭详情页
/// @param nativeExpressAd 广告
- (void)ptg_nativeExpressAdVDidCloseOtherController:(PTGNativeExpressAd *)nativeExpressAd {
    NSLog(@"信息流广告详情页被关闭");
}

```

banner广告

Banner广告：是在内容底部或顶部显示的小条形广告。支持的广告位宽高比：2:1、3:2、6:5、30:13、20:3、4:1、6.4:1、345:194共8个尺寸，开发者按照展示场景进行勾选。创建好的尺寸不支持修改。

banner广告加载

banner广告加载示例：

```

#import <PTGAdSDK/PTGAdSDK.h>

@interface PTGNativeExpressBannerViewController ()<PTGNativeExpressBannerAdDelegate>

```

```

@property(n nonatomic, strong)PTGNativeExpressBannerAd *bannerAd;

@end

@implementation PTGNativeExpressBannerViewController

- (void)viewDidLoad {
    [super viewDidLoad];
    [self.bannerAd loadAd];
}

- (PTGNativeExpressBannerAd *)bannerAd {
    if (!_bannerAd) {
        _bannerAd = [[PTGNativeExpressBannerAd alloc] initWithPlacementId:@"900000229" size:CGSizeMake(s
        _bannerAd.delegate = self;
        _bannerAd.rootViewController = self;
    }
    return _bannerAd;
}

@end

```

Banner广告事件

设置banner广告的delegate，delegate遵守并实现PTGNativeExpressBannerAdDelegate，可以监听广告的生命周期事件。

在banner广告加载成功后，调用PTGNativeExpressBannerAd对象的showAdFromView:(UIView *)view frame:(CGRect)frame方法，传入需要展示广告的视图，以及广告视图的frame，进行banner广告的展示。PTGNativeExpressBannerAd对象的realSize属性，用于设置banner广告的大小，代码示例如下：

```

#pragma mark - PTGNativeExpressBannerAdDelegate -
/// 广告加载成功
/// 在此方法中调用 showAdFromView:frame 方法
- (void)ptg_nativeExpressBannerAdDidLoad:(PTGNativeExpressBannerAd *)bannerAd {
    NSLog(@"横幅广告加载成功%@", bannerAd);
    /// 广告是否有效 (展示前请务必判断)
    /// 如不严格按照此方法对接，将导致因曝光延迟时间造成的双方消耗gap过大，请开发人员谨慎对接
    if (bannerAd.isReady) {
        [bannerAd showAdFromView:self.view frame:(CGRect){0, 200}, self.bannerAd.realSize];
    } else {

```

```

        NSLog(@"广告已过期");
    }
}

///  
广告加载失败
- (void)ptg_nativeExpressBannerAd:(PTGNativeExpressBannerAd *)bannerAd didLoadFailWithError:(NSError *_N
    NSLog(@"横幅广告加载失败%@", error);
}

///  
广告将要曝光
- (void)ptg_nativeExpressBannerAdWillBeComVisible:(PTGNativeExpressBannerAd *)bannerAd {
    NSLog(@"横幅广告曝光%@", bannerAd);
}

///  
广告曝光失败
- (void)ptg_nativeExpressBannerAdBeComVisibleFail:(PTGNativeExpressBannerAd *)bannerAd error:(NSError *_N
    NSLog(@"横幅广告曝光失败%@", error = %@", bannerAd, error);
}

///  
广告被点击
- (void)ptg_nativeExpressBannerAdDidClick:(PTGNativeExpressBannerAd *)bannerAd {
    NSLog(@"横幅广告被点击%@", bannerAd);
}

///  
广告被关闭
- (void)ptg_nativeExpressBannerAdClosed:(PTGNativeExpressBannerAd *)bannerAd {
    NSLog(@"横幅广告被关闭%@", bannerAd);
}

///  
广告详情页给关闭
- (void)ptg_nativeExpressBannerAdViewDidCloseOtherController:(PTGNativeExpressBannerAd *)bannerAd {
    NSLog(@"横幅广告详情页被关闭%@", bannerAd);
}

```

插屏广告

插屏广告是移动广告的一种常见形式，在应用流程中弹出，当应用展示插屏广告时，用户可以选择点击广告，访问其目标网址，也可以将其关闭并返回应用。在应用执行流程的自然停顿点，适合投放这类广告。

插屏广告加载

插屏广告加载示例：


```

#import <PTGAdSDK/PTGAdSDK.h>

@interface PTGNativeExpressInterstitialAdViewController ()<PTGNativeExpressInterstitialAdDelegate>

@property(nonatomic,strong)PTGNativeExpressInterstitialAd *interstitialAd;
@end

@implementation PTGNativeExpressInterstitialAdViewController

- (void)viewDidLoad {
    [super viewDidLoad];
    [self.interstitialAd loadAd];
}

- (PTGNativeExpressInterstitialAd *)interstitialAd {
    if (!_interstitialAd) {
        _interstitialAd = [[PTGNativeExpressInterstitialAd alloc] initWithPlacementId:@"900000230"];
        _interstitialAd.adSize = CGSizeMake(300, 300);
        _interstitialAd.delegate = self;
    }
    return _interstitialAd;
}
@end

```

插屏广告事件

设置插屏广告的delegate，delegate遵守并实现PTGNativeExpressInterstitialAdDelegate，可以监听广告的生命周期事件。

在插屏广告加载成功后，调用PTGNativeExpressInterstitialA实例的showAdFromRootViewController方法，展示插屏广告，

代码示例如下：

```

#pragma mark - PTGNativeExpressInterstitialAdDelegate -

- (void)ptg_nativeExpresInterstitialAdDidLoad:(PTGNativeExpressInterstitialAd *)interstitialAd {
    NSLog(@"插屏广告加载成功%@", interstitialAd);
    /// 广告是否有效 (展示前请务必判断)
    /// 如不严格按照此方法对接, 将导致因曝光延迟时间造成的双方消耗gap过大, 请开发人员谨慎对接
    if (interstitialAd.isReady) {
        [interstitialAd showAdFromRootViewController:self];
        NSLog(@"广告展示中");
    } else {

```

```

        NSLog(@"广告已过期");
    }
}

- (void)ptg_nativeExpressInterstitialAd:(PTGNativeExpressInterstitialAd *)interstitialAd didFailWithError
    NSLog(@"插屏广告加载失败%@", error);
}

- (void)ptg_nativeExpressInterstitialAdWillVisible:(PTGNativeExpressInterstitialAd *)interstitialAd {
    NSLog(@"插屏广告曝光%@", interstitialAd);
}

- (void)ptg_nativeExpressInterstitialAdVisibleFail:(PTGNativeExpressInterstitialAd *)interstitialAd error
    NSLog(@"插屏广告展示失败 error = %@", error);
}

- (void)ptg_nativeExpressInterstitialAdDidClick:(PTGNativeExpressInterstitialAd *)interstitialAd {
    NSLog(@"插屏广告被点击%@", interstitialAd);
}

- (void)ptg_nativeExpressInterstitialAdDidClose:(PTGNativeExpressInterstitialAd *)interstitialAd {
    NSLog(@"插屏广告被关闭%@", interstitialAd);
}

- (void)ptg_nativeExpressInterstitialAdDidCloseOtherController:(PTGNativeExpressInterstitialAd *)intersti
    NSLog(@"插屏广告详情页被关闭%@", interstitialAd);
}

- (void)ptg_nativeExpressInterstitialAdDisplayFail:(PTGNativeExpressInterstitialAd *)interstitialAd error
    NSLog(@"插屏广告展示失败%@", interstitialAd);
}

```

激励广告

激励广告将短视频融入到APP场景当中，用户观看短视频广告后可以给予一些应用内奖励。常出现在游戏的复活、任务等位置，或者网服类APP的一些增值服务场景。

激励广告加载

激励广告加载示例：

```
#import <PTGAdSDK/PTGAdSDK.h>
```

```

@interface PTGNativeExpressRewardVideoAdViewController ()<PTGRewardVideoDelegate>

@property(nonatomic, strong)PTGNativeExpressRewardVideoAd *rewardVideoAd;

@end

@implementation PTGNativeExpressRewardVideoAdViewController

- (void)viewDidLoad {
    [super viewDidLoad];
    [self.rewardVideoAd loadAd];
}

- (PTGNativeExpressRewardVideoAd *)rewardVideoAd {
    if (!_rewardVideoAd) {
        PTGRewardedVideoModel *model = [[PTGRewardedVideoModel alloc] init];
        model.userId = @"user id";
        model.rewardName = @"奖励名称";
        model.rewardAmount = 400;
        _rewardVideoAd = [[PTGNativeExpressRewardVideoAd alloc] initWithPlacementId:@"900000400" rewardedVideoModel:model];
        _rewardVideoAd.delegate = self;
    }
    return _rewardVideoAd;
}

@end

```

激励广告事件

设置激励广告的delegate，delegate遵守并实现PTGRewardVideoDelegate，可以监听广告的生命周期事件。

在激励广告加载成功后，调用PTGNativeExpressRewardVideoAd实例的showAdFromRootViewController方法，展示激励广告，

```

#pragma mark - PTGRewardVideoDelegate -

/// 激励广告加载成功
/// 在此方法中调用showAdFromRootViewController 展示激励广告
- (void)ptg_rewardVideoAdDidLoad:(PTGNativeExpressRewardVideoAd *)rewardVideoAd {
    NSLog(@"激励广告加载成功%@", rewardVideoAd);
}

/// 激励广告失败 加载失败 播放失败 渲染失败
- (void)ptg_rewardVideoAd:(PTGNativeExpressRewardVideoAd *)rewardVideoAd didFailWithError:(NSError *)error {
}

```

```

        NSLog(@"激励广告加载失败%@", error);
    }

    /// 激励广告将要展示
    - (void)ptg_rewardVideoAdWillVisible:(PTGNativeExpressRewardVideoAd *)rewardVideoAd {
        NSLog(@"激励广告将要展示%@", rewardVideoAd);
    }

    /// 激励广告曝光
    - (void)ptg_rewardVideoAdDidExposed:(PTGNativeExpressRewardVideoAd *)rewardVideoAd {
        NSLog(@"激励广告曝光%@", rewardVideoAd);
    }

    - (void)ptg_rewardVideoAdExposedFail:(PTGNativeExpressRewardVideoAd *)rewardVideoAd error:(NSError *)err {
        NSLog(@"激励广告曝光失败 error = %@", error);
    }

    /// 激励广告关闭
    - (void)ptg_rewardVideoAdDidClose:(PTGNativeExpressRewardVideoAd *)rewardVideoAd {
        NSLog(@"激励广告关闭%@", rewardVideoAd);
    }

    /// 激励广告被点击
    - (void)ptg_rewardVideoAdDidClicked:(PTGNativeExpressRewardVideoAd *)rewardVideoAd {
        NSLog(@"激励广告被点击%@", rewardVideoAd);
    }

    /// 激励广告播放完成
    - (void)ptg_rewardVideoAdDidPlayFinish:(PTGNativeExpressRewardVideoAd *)rewardVideoAd {
        NSLog(@"激励广告播放完成%@", rewardVideoAd);
    }

    - (void)ptg_rewardVideoAdDidRewardEffective:(PTGNativeExpressRewardVideoAd *)rewardedVideoAd {
        NSLog(@"激励广告达到激励条件%@", rewardedVideoAd);
    }
}

```

其他

SDK 支持媒体决定是否启用摇一摇广告，默认启用

```

/// YES 为启用，默认启用，NO不启用摇一摇广告形式
[PTGSDKManager setSensorStatus:YES];

```

错误码	描述
10001	没有广告
10002	广告数据格式异常
10003	无效的配置数据（聚合配置）
10004	广告请求超时
10005	网络错误 / 无网络
10006	初始化失败
10007	未初始化
10008	广告位 ID 为空
10009	广告位 ID 不合法
10010	广告初始化参数不合法，比如 size
10011	广告加载异常，第三方消耗方广告加载失败
10012	没有广告容器
10013	代码中没有对应消耗方，适配器为空
10014	资源加载错误
10015	渲染错误
10016	广告过期
10017	广告展示异常
10018	广告视频播放失败
10019	内部错误
10020	未知错误
10021	不支持的类型
10022	不支持的 DP
10023	缺少请求参数
10024	没有合适的策略
10025	包名校验错误
10026	广告价格低于底价
10027	广告 Style 不合法
10028	广告位 id 所属类型和广告 API 类型不匹配

错误码	描述
10029	请求 URL 为空
10030	POST data 为空
10031	AES 密钥为空
10032	URL 格式错误
10033	接口返回数据为空
10034	AES 解密参数为空
10035	请求响应格式错误
10036	AES 解密失败 / 异常
10037	AES 加密参数为空
10038	AES 加密失败 / 异常
10039	序列化异常
10040	广告被策略过滤，不允许展示
10041	Banner PreRenderView 不存在
10042	传入的 VC 不存在
10043	有 presentedViewController 无法展示
10044	不支持的广告形式
10045	下载 apk 超时